

ALGORITHMIQUE ET PROGRAMMATION

Pr Amadou Dahirou GUEYE

Enseignant-chercheur en Informatique, UFR STA - UAM

Contact : dahirou.gueye@uam.edu.sn

Chapitre 3 : Les tableaux

1 Notion de tableau

Soit un entier n positif.

Supposons qu'on veuille saisir n valeurs réelles afin de calculer leur moyenne, ou de trouver leur minimum, ou de les afficher par ordre croissant.

Pour des valeurs petites de n , on peut déclarer n variables réelles pour résoudre le problème. Mais si n est assez grand, on se rend compte que cela devient impropre, fastidieux, voire impossible.

Il faudrait, dans ce cas, utiliser une variable permettant de représenter les n valeurs. Le type de données de cette variable serait le type tableau.

Définition :

Un tableau est une collection séquentielle d'éléments de même type, où chaque élément peut être identifié par sa position dans la collection. Cette position est appelée indice et doit être de type scalaire.

Déclaration :

Pour déclarer un tableau, il faut donner :

- son nom (identificateur de la variable)
- Ses bornes: la borne inférieure correspondante à l'indice minimal et la borne supérieure correspondante à l'indice maximal.
- le type des éléments le composant.

Syntaxe:

type nom=**tableau** [<indice minimum> .. <indice maximum>] **de**
<type des composants>

ou

variable nom : **tableau** [<indice minimum> .. <indice maximum>]
de <type des composants>

Exemple:

Variable t : tableau [1 .. 10] de réels

En pascal, on a:

Syntaxe:

type nom=**array** [<indice minimum> .. <indice maximum>] **of** <type des composants>

ou

var nom : **array** [<indice minimum> .. <indice maximum>] **of** <type des composants>

Exemple:

Variable t : array [1 .. 10] of real;

Schématiquement, on va représenter la variable t comme suit :

1	2	3	4	5	6	7	8	9	10
8.4	3.5	12	20	10	13.34	50	100	30.1	60.9

Dans la mémoire centrale, les éléments d'un tableau sont stockés de façon linéaire, dans des zones contiguës.

Le tableau ci-dessus est de dimension 1, nous verrons un peu plus loin que l'on peut représenter des tableaux à 2 dimensions, voire même plus.

L'élément n° I sera représenté par l'expression $t[I]$.

Dans notre exemple, $t[I]$ peut être traité comme une variable réelle. On dit que le tableau t est de taille 10.

2 Création d'un tableau

La création d'un tableau consiste au remplissage des cases le composant. Cela peut se faire par saisie, ou par affectation.

Par exemple, pour remplir le tableau t précédent, on peut faire :

$t[1] := 8.4$; $t[2] := 3.5$; ... $t[10] := 60.9$;

Si on devait saisir les valeurs, il faudrait écrire :

Pour $i := 1$ à 10 faire

 lire($t[i]$) ;

3 Affichage d'un tableau

Afficher un tableau revient à afficher les différents éléments qui le composent. Pour cela, on le parcourt (généralement à l'aide d'une boucle avec compteur) et on affiche les éléments un à un.

Exercice d'application

Ecrire un programme qui permet de créer un tableau d'entiers t1 de taille 20 par saisie, et un tableau t2 de même taille en mettant dans t2[i] le double de t1[i], $i \in \{1, \dots, 20\}$.

```
Type tab = array[1..20] of integer ;
```

```
var t1, t2 : tab ;
```

```
  i : integer ;
```

```
begin
```

```
  { saisie de t1 }
```

```
  for i :=1 to 20 do
```

```
    begin
```

```
      write('donner t1[' ,i ,'] : ');
```

```
      readln(t1[i]);
```

```
    end;
```

```
  { création de t2 }
```

```
  for i :=1 to 20 do
```

```
    t2[i] := 2*t1[i];
```

```
  { affichage de t2 }
```

```
  for i :=1 to 20 do
```

```
    write('t2[' ,i ,'] = ', t2[i]);
```

```
end.
```

4 Traitement d'un tableau

Après avoir créé un tableau, on peut y effectuer plusieurs opérations comme le calcul de la somme ou de la moyenne des éléments, la recherche du plus petit ou du grand élément du tableau, le test d'appartenance d'un objet au tableau, ...

Pour la suite, on considère un tableau d'entiers t déclaré comme suit :

```
Var t : array[1 .. n] of integer ;
```

1. Somme des éléments d'un tableau

On effectue la somme des éléments du tableau t, le résultat est dans la variable S :

```
S := 0 ;
```

```
for i :=1 to n do S := S + t[i];
```

```
writeln('la somme des éléments de t est:', S);
```

2. Minimum d'un tableau

On cherche le plus petit élément du tableau t, le résultat est dans la variable min :

```
min := t[1] ;
```

```
for i :=2 to 10 do
```

```
if t[i] < min then min := t[i];
```

```
writeln('Le minimum des elements de t est:', min) ;
```

4.3 Test d'appartenance

On cherche si l'entier x appartient à t , le résultat est mis dans la variable booléenne appartient :

```
appartient := false
```

```
for i:=1 to n do
```

```
  if (t[i]=x) then
```

```
    appartient := true;
```

```
  if (appartient) then
```

```
    write('x appartient à t')
```

```
  else write('x n'appartient pas à t');
```

On remarque que l'on peut arrêter les itérations (la recherche) si l'on rencontre l'élément x dans t . Pour cela, il faut utiliser une boucle *Tant Que* ou *Repeat*:

```
i := 1;
```

```
while (i<=n) and (t[i]<>x) do
```

```
  i:=i+1;
```

```
if (i>n) then
```

```
  write('x n'appartient pas à t')
```

```
else
```

```
  write('x appartient à t');
```



```

i := 0 ;
Repeat
    i:=i+1;
Until(i>n) or (t[i]=x) ;

```

```

if i>n then
    write('x n"appartient pas à t')
else write('x appartient à ');

```

Dans les deux cas, si x appartient à t , la valeur de i est l'indice de la case qui le contient.

4.4 Ajout d'un élément

On suppose que le tableau t est « rempli » et qu'il reste une case non occupée à la fin (la $n+1^{\text{ième}}$ case). (n étant le nombre d'éléments effectif du tableau différent de la taille maximale)

Si on veut alors ajouter un entier x à la fin du tableau, il suffira d'écrire

```
t[n+1] := x ;
```

Mais, si on veut ajouter x dans une case dont l'indice k est différent de $n+1$, il faudra décaler les éléments $t[k]$, $t[k+1]$, ..., $t[n]$ vers la droite pour libérer la case d'indice k :

```
for i := n+1 downto k+1 do
```

```
t[i] := t[i-1];
```

```
t[k] := x;
```

4.5 Suppression d'un élément

Pour supprimer l'élément se trouvant à la position k de t , on l'écrase en faisant décaler les éléments placés après lui vers la gauche :

```
for  $i := k$  to  $n-1$  do
```

```
   $t[i] := t[i+1];$ 
```

Il faut noter qu'après une telle opération, l'élément se trouvant à la dernière position (n) n'est plus significatif.

5 Tableaux de caractères

1. Notion de chaîne de caractères

Une chaîne de caractères est soit une chaîne vide, soit un caractère suivi d'une chaîne de caractères ; en un mot c'est une collection de caractères.

Exemples : "Bonjour", "L'UAM se situe à Diamniadio", "A", "2023"

La plupart des langages de programmation, Pascal notamment, disposent d'un type chaîne de caractères et d'un ensemble de fonctions prédéfinies permettant de traiter les chaînes de caractères. Ces fonctions permettent de calculer la longueur d'une chaîne, de concaténer deux chaînes, d'extraire une sous-chaîne, de comparer deux chaînes, etc. Il faut noter qu'une chaîne de caractères peut être traitée comme un tableau de caractères.

2. Le type STRING de Pascal

En Pascal, une variable de type STRING est une séquence de caractères de longueur variable au cours de l'exécution et une taille prédéfinie entre 1 et 255.

Exemple :

```
var s : string ; (* 255 caractères sont alors réservés *)  
    s10 : string[10] ; (* seuls 10 caractères sont réservés *)
```

On peut appliquer les opérateurs suivants sur des variables de type STRING :

+, =, <>, <=, >=, <, >.

Les fonctions prédéfinies les plus usuelles sont :

- length(s : STRING) : integer;
- retourne la longueur courante de s. copy(s :STRING ;p :integer ; l : integer): STRING;
renvoie une chaîne constituée de **l** caractères à partir de la position **p**.
- concat (s1,s2 : STRING): STRING ; renvoie la chaîne résultant de la concaténation de s1 et de s2.
- pos (ssch : STRING ; ch : STRING): Byte;
renvoie la position du premier caractère de la sous-chaîne **ssch** dans la chaîne **ch** ;
si la sous-chaîne ne se trouve pas dans la chaîne, elle renvoie 0.

6 Les tableaux à deux dimensions

Pour traiter les notes obtenues par un étudiant à 10 épreuves on peut utiliser un tableau de 10 réels.

Pour traiter les notes obtenues par 5 étudiants, on pourrait utiliser 5 tableaux de 10 réels chacun. Mais puisqu'on va effectuer très probablement les mêmes traitements sur ces tableaux, il est préférable de les regrouper dans une seule variable qui sera un tableau de 5 lignes et 10 colonnes. Chaque élément de ce tableau multidimensionnel sera identifié par deux indices : la position indiquant la ligne et la position indiquant la colonne.

Déclaration :

variable t : tableau[1..5, 1..10] de reels ;

En pascal, on aurait

var t : array[1..5, 1..10] of real ;

Pour accéder à l'élément se trouvant sur la ligne i et la colonne j, on utilise le terme **t[i,j]**.

Exercice d'application

Ecrire un programme qui permet de créer (par saisie) et d'afficher un tableau t à deux dimensions d'entiers de taille 3x5.

```
Type TAB_3_5 = array[1..3, 1..5] of integer ;
```

```
var t : TAB_3_5 ; i,j : integer ;
```

```
begin
```

```
  (* saisie de t *)
```

```
for i :=1 to 3 do
  for j :=1 to 5 do begin
    write('donner t[',i,',',j,']:');
    readln(t[i]); end;
(* affichage de t *)
for i :=1 to 3 do
begin
  for j :=1 to 5 do write(t[i,j], ' ');
  writeln; end;
end.
```

Remarque : Les ensembles en Pascal

Le langage Pascal permet la définition de types « ensembles » à l'aide du mot réservé **set**.
L'expression **set of T** définit un type ensemble dont les éléments sont de type **T**. **T** est appelé le type de base de l'ensemble et doit être un type scalaire comportant moins de 256 valeurs ; les bornes doivent s'inscrire entre 0 et 255.

Exemple de définition de types ensembles:

TYPE

EnsCar = set of char ;

Chiffre = set of 0 .. 9; (* 0 .. 9 est appelé sous-intervalle *)

JOUR = (dim, lun, mar, mer, jeu, ven, sam) ; (* JOUR est un type énuméré *)

Jours = set of JOUR ;

7 Exercices proposés

Exercice 1:

Ecrire un programme qui permet de saisir des nombres entiers dans un tableau à deux dimensions TAB (10, 20) et de calculer les totaux par ligne et par colonne dans des tableaux TOTLIG(10) et TOTCOL(20).

Exercice 2 :

Un palindrome est un mot ou une phrase qui se peut se lire de la même façon de gauche à droite et de droite à gauche. Par exemple:

"ANNA" et "ESOPE RESTE ICI ET SE REPOSE".

Proposer un programme qui permet de dire si un mot ou une phrase est un palindrome.

Exercice 3 :

On considère un tableau t (n,m) à deux dimensions.

Ecrire un programme qui calcule la somme et le produit des éléments de t .

Ecrire un programme qui affiche le plus petit élément de t et sa position.

Exercice 4 :

Ecrire un programme qui met à zéro les éléments d'un tableau à deux dimensions tels que l'indice de ligne est égal à l'indice de colonne.

Exercice 5 :

Ecrire un programme qui permet de chercher une valeur x dans un tableau à deux dimensions $t(m,n)$. Le programme doit aussi afficher les indices ligne et colonne si x a été trouvé.